

STEP-BY-STEP RUNNING GUIDE

SmartShield IPS

Linux + Docker · Demo Recording Edition

Version 1.1.16 · Factory Build

Written in plain language — no prior experience needed.

Prepare → Run → Simulate Attacks → See Alerts → Record Demo

What you'll do in this guide: stand up SmartShield on your own Linux PC, pretend your PC is a factory network, simulate OT attacks with simple scripts, and show the alerts appearing live on the web interface.

Good luck with the demo!

What Is SmartShield?

Think of SmartShield like a smart security guard for a factory. In a real factory, machines talk to each other over a network — PLCs (the computers that control motors, valves, conveyors) send messages back and forth constantly. SmartShield sits quietly and watches all these messages. If someone tries to attack the machines — sending fake commands, flooding the network, or trying to stop a PLC — SmartShield catches it, logs it, and can even block the attacker.

Your system has two parts:

- **SmartShield IPS:** the brain. Runs as a Python program inside Docker.
- **Web interface:** a webpage you open in your browser to see alerts in real time.

For your demo

You'll run SmartShield on your own Linux PC, pretend it's a factory network, simulate attacks (using simple scripts), and show alerts appearing live on the web interface — exactly what an examiner wants to see.

New in this build (v1.1.16) — good things to mention on camera

- **SYN floods now get caught.** The new single-connection volumetric detector flags a `hping3 --flood` attack that rides one connection — earlier builds could miss it.
- **Separate S7 master list.** S7Comm unauthorized-access checks use their own `s7_authorized_masters.conf` now.
- **Newer base:** Zeek 8, a patched Redis, and faster evidence handling.

What You Need Before Starting

Check your Linux has these installed

Open a terminal and run each command. If it prints a version number, you're good. If it says "command not found", install it.

```
# Check Docker
docker --version           # e.g. Docker version 24.0.5
# Check Docker Compose
docker compose version     # e.g. Docker Compose version v2.20.0
# Check Python
python3 --version          # Python 3.10 or higher
# Check Git (optional)
git --version
```

If Docker is not installed:

```
sudo apt update
sudo apt install -y docker.io docker-compose-plugin
sudo systemctl start docker
sudo systemctl enable docker
# Allow your user to run Docker without sudo:
sudo usermod -aG docker $USER
# IMPORTANT: log out and log back in after this command
```

Install the attack-simulation tools

```
sudo apt install -y nmap hping3 netcat-openbsd
# Python Modbus simulator (to simulate PLC attacks)
pip3 install pymodbus scapy --break-system-packages
```

What these tools do

- **nmap** — scans ports, like a burglar checking which windows are open.
- **hping3** — sends many packets very fast, like flooding a factory gate with trucks.
- **pymodbus** — speaks the Modbus language PLCs use, so we can fake PLC commands.
- **netcat** — sends raw data to any port, like knocking on a door very fast.

Steps to Run SmartShield

1 Get the project code ready

You already have the zip file (`SmartShield_newVersion.zip`). Put it somewhere easy to find, like your home folder.

```
cd ~
# Extract into a clean folder named SmartShield-Factory:
unzip SmartShield_newVersion.zip -d SmartShield-Factory
cd SmartShield-Factory
ls
# You should see: modules/ config/ docker/ webinterface/ zeek-scripts/ install/ ...
```

Good sign

If you see folders like `modules config docker webinterface zeek-scripts` you're in the right place.

2 Set up the configuration files

Tell SmartShield which IPs to protect and which are authorized. For a local demo we use fake local IPs.

2a. Create a simple OT device protection list

```
nano config/ot_protected_devices.conf
# Add these lines (pretend these are your factory PLCs), then Ctrl+O, Ctrl+X:
172.20.0.10 # Fake PLC-01
172.20.0.11 # Fake PLC-02
172.20.0.20 # Fake HMI
```

2b. Leave the authorized-master files empty for now

An empty masters file means SmartShield treats *all* writes as unauthorized — perfect for showing attacks in the demo. The build now has two master files; leaving both empty is fine.

```
ls config/modbus_authorized_masters.conf
ls config/s7_authorized_masters.conf # NEW in v1.1.16 – leave empty too
```

2c. Set your web interface password

```
cat > docker/.env << 'EOF'
SMARTSHIELD_INTERFACE=eth0
SMARTSHIELD_WEB_USER=admin
SMARTSHIELD_WEB_PASSWORD=Demo1234567!
SMARTSHIELD_SECRET_KEY=abc123def456ghi789jkl012mno345pqr678
EOF
echo "Done! .env file created."
```

Important for the demo

The interface name (`eth0`) doesn't matter much for a local demo — SmartShield reads from the network or a PCAP. For live capture, replace `eth0` with your real interface name. Find it with `ip link show`.

3 Build the Docker image

This turns the code into a ready-to-run container. You only do this once (or when the code changes). The first build can take 5–15 minutes — go make tea.

```
cd ~/SmartShield-Factory
docker build -f docker/Dockerfile -t smartshield_gp:latest .
# When it finishes you should see: Successfully tagged smartshield_gp:latest
```

If the build fails with a network timeout, just run it again (optionally with `--no-cache`). Verify the image exists:

```
docker images | grep smartshield
# smartshield_gp latest abc123... 5 minutes ago 1.2GB
```

4 Create the data folders

SmartShield saves alerts and Redis data on your computer so they survive restarts.

```
sudo mkdir -p /var/lib/smartshield/{redis,output}
sudo chmod 777 /var/lib/smartshield/{redis,output}
ls /var/lib/smartshield/ # should show: output redis
```

5 Start SmartShield

One command starts everything — SmartShield, Redis (the database), and the web interface — on an isolated Docker bridge network.

```
cd ~/SmartShield-Factory
docker compose -f docker/docker-compose.factory.yml up -d
# -d means "run in background" so your terminal stays usable

sleep 10
docker compose -f docker/docker-compose.factory.yml ps
```

You should see three containers up:

NAME	STATUS	PORTS
smartshield	Up 15 seconds	
smartshield-redis	Up 15 seconds	6379/tcp
smartshield-web	Up 15 seconds	0.0.0.0:55000->55000/tcp

If a container shows "Exiting" or "Error"

Run `docker logs smartshield` and `docker logs smartshield-redis`. The most common cause is a missing `docker/.env` file — check Step 2c.

6 Open the web interface

The web interface is where you'll watch alerts during the demo. Open it in your browser:

```
http://localhost:55000
# Or from a Linux desktop: xdg-open http://localhost:55000
```

You'll see a login page. Enter:

Field	Value
Username	admin
Password	Demo1234567! (or whatever you set in Step 2c)

You're in

After login you'll see the dashboard. It looks empty at first — that's normal. Alerts appear once attacks are detected. Keep this tab open for the demo.

To watch live logs in a second terminal:

```
docker logs smartshield -f      # -f follows new lines in real time; Ctrl+C to stop
```

How to Test It — Simulating Attacks

For the demo you'll run fake attacks against your own computer. SmartShield is watching the network and will detect them — like a fire drill, you set off the alarm on purpose to prove the system works.

These attacks are safe on your own PC

Every command below targets `localhost` (127.0.0.1) or your own machine. **Never** run these against a real factory network or anyone else's network.

Attack 1 — Port scan (a hacker discovering the network)

A port scan is like a burglar walking down a street knocking on every door. SmartShield sees many connections and flags it.

```
# In a new terminal:
nmap -sS -p 502,102,20000 127.0.0.1
nmap -sV -p 1-1000 127.0.0.1
nmap -p 502,102,20000,4840,44818,34964 --open 127.0.0.1
```

Check the web interface — a new IP profile should appear within a minute or two.

Attack 2 — Modbus connection flood (flooding a PLC)

Like calling a PLC's phone thousands of times a minute so the real operator can't get through. SmartShield counts the connections and alerts when the threshold is crossed.

First, run a fake Modbus PLC so there's something to attack:

```
# Terminal 1 – fake Modbus PLC server
sudo python3 - << 'PYEOF'
from pymodbus.server import StartTcpServer
from pymodbus.datastore import ModbusSlaveContext, ModbusServerContext
from pymodbus.datastore import ModbusSequentialDataBlock
store = ModbusSlaveContext(
    di=ModbusSequentialDataBlock(0, [17]*100),
    co=ModbusSequentialDataBlock(0, [17]*100),
    hr=ModbusSequentialDataBlock(0, [17]*100),
    ir=ModbusSequentialDataBlock(0, [17]*100))
context = ModbusServerContext(slaves=store, single=True)
print("Fake PLC running on port 502...")
StartTcpServer(context=context, address=("0.0.0.0", 502))
PYEOF
```

Then flood it from a second terminal:

```
# Terminal 2 – connection flood
python3 - << 'PYEOF'
from pymodbus.client import ModbusTcpClient
print("Starting Modbus flood attack...")
for i in range(50):
    try:
        c = ModbusTcpClient("127.0.0.1", port=502)
        c.connect(); c.read_coils(0, 10); c.close()
    except Exception:
        pass
    if i % 10 == 0:
        print(f"Sent {i} connections...")
print("Done! Check the SmartShield web interface.")
PYEOF
```

What to show on camera

Web interface → Analysis page. The attacker IP (127.0.0.1) appears with evidence of a Modbus connection flood at **HIGH** threat level.

Attack 3 — Modbus unauthorized write (tampering with a PLC)

The most dangerous attack in real factories: an unauthorized computer sends a WRITE command that could open a valve or stop a motor. SmartShield flags writes from non-authorized sources.

```
# Keep the fake PLC from Attack 2 running in Terminal 1.
# Terminal 2 – unauthorized write:
python3 - << 'PYEOF'
from pymodbus.client import ModbusTcpClient
print("Attempting unauthorized Modbus writes...")
c = ModbusTcpClient("127.0.0.1", port=502)
if c.connect():
    print(c.write_coil(0, True))           # func 5 – single coil
    print(c.write_coils(0, [True,False,True,True])) # func 15 – multiple coils
    print(c.write_register(1, 9999))       # func 6 – single register
    c.close(); print("Attack complete. Check SmartShield!")
else:
    print("Could not connect. Is the fake PLC running?")
PYEOF
```

What this shows the examiner

SmartShield detects unauthorized writes to PLCs — the most dangerous ICS attack. The evidence type is **OT_MODBUS_WRITE_COILS** at **CRITICAL** threat level. In a real factory this would page the SOC.

Attack 4 — TCP SYN flood (denial of service on the OT network)

A flood overwhelms a device so it stops responding — like calling an emergency line a thousand times a second. **This is the attack the new v1.1.16 detector was built for.**

```
# SYN flood on Modbus port 502 using hping3
sudo hping3 -S -p 502 --flood 127.0.0.1
# Let it run ~10 seconds, then Ctrl+C to stop
```

Highlight this one on camera

A **--flood** attack rides a single long-lived connection, which older builds logged as just one flow and could miss. The new **single-connection volumetric detector** watches packet-rate inside one flow and fires **OT_MODBUS_CONNECTION_FLOOD** at **HIGH** (0.90 confidence). Mention this is a v1.1.16 improvement.

Attack 5 — Nmap service scan (network discovery / reconnaissance)

Attackers scan to learn what's on the network before striking — the first stage of any cyberattack.

```
nmap -sV -O 127.0.0.1
sudo nmap -sU -p 20000,34964,319,123 127.0.0.1
nmap -A -p 502,102 127.0.0.1
```

Run order for maximum visual impact

1. Nmap scan first (easy to explain)
2. Modbus flood (dramatic spike in connections)

3. SYN flood (call out the new volumetric detector)
4. Unauthorized write (most serious — the CRITICAL alert)
5. Show the web interface filling with alerts, then `blocking.log`

Seeing the Results

In the web interface

Page	What it shows
Overview (/)	Summary — how many profiles, how many alerts, when analysis started
Analysis (/analysis)	The main page — click any IP to see all evidence and attack detail
Timeline	A chronological view of all events — great for the examiner

In the terminal

```
docker logs smartshield -f                # live logs
cat /var/lib/smartshield/output/alerts.json # the alerts file
watch -n 1 "tail -20 /var/lib/smartshield/output/alerts.json"

# What IPs are blocked:
sudo iptables -L smartshieldBlocking -n 2>/dev/null || echo "blocking module not active"

# Counts from Redis (runs inside the redis container):
docker exec smartshield-redis redis-cli KEYS "alert_*" | wc -l
docker exec smartshield-redis redis-cli KEYS "evidence_*" | head -20
```

One-shot health check

```
echo "=== Containers ===" && docker compose -f docker/docker-compose.factory.yml ps
echo "=== DB size ==="      && docker exec smartshield-redis redis-cli DBSIZE
echo "=== Output files ===" && ls -lh /var/lib/smartshield/output/
echo "=== Redis persisting? ===" && docker exec smartshield-redis redis-cli INFO persistence |
grep aof_enabled
```

Stopping and Restarting

```
# Stop (data is saved to /var/lib/smartshield/):
docker compose -f docker/docker-compose.factory.yml down

# Start again later:
cd ~/SmartShield-Factory
docker compose -f docker/docker-compose.factory.yml up -d

# Restart after a config change:
docker compose -f docker/docker-compose.factory.yml restart
docker restart smartshield      # or just SmartShield
```


Recording a Good Demo

Recommended script (8–10 minutes)

Time	Section	What to say / show
0:00–1:00	Intro	Show the empty web interface. Explain what SmartShield is and where it sits in a factory network.
1:00–2:00	Start monitoring	Show <code>docker logs smartshield -f</code> running. Explain Zeek is capturing traffic.
2:00–3:30	Attack 1: port scan	Run nmap. Switch to the web interface — a new IP profile appears.
3:30–5:00	Attack 2: Modbus flood	Start the fake PLC, run the flood. Show the connection count rising.
5:00–6:00	Attack 4: SYN flood	Run hping3. Call out that the new volumetric detector catches single-connection floods.
6:00–7:30	Attack 3: unauthorized write	Run the write attack. Show the CRITICAL alert. Explain it would stop a motor or open a valve.
7:30–8:30	Show blocking + OT-safe	Run <code>iptables -L smartshieldBlocking -n</code> . Explain a PLC IP would never be auto-blocked.
8:30–10:00	SIEM export	Show <code>alerts.json</code> . Explain it feeds Splunk / QRadar / Wazuh.

Screen recording tips

- Use OBS Studio or SimpleScreenRecorder: `sudo apt install obs-studio simplescreenrecorder`
- Record at 1080p. Put two windows side by side: terminal left, browser right.
- Zoom into the alerts section when a CRITICAL alert appears — the red is very visible.
- Narrate as you go, and show the alert timestamps to prove detection is real-time.

What to say at each moment

Moment	What to say
Showing the web UI	SmartShield builds a behavioral profile for every IP and alerts when behavior deviates from normal.
Showing a Modbus alert	Modbus is the most common industrial protocol. An unauthorized coil write could open a safety valve or stop a line.
Showing the SYN flood alert	This build adds single-connection volumetric detection, so a <code>--flood</code> DoS that hides in one connection still gets caught.
Showing OT-safe blocking	Blocking a PLC IP would stop production, so SmartShield alerts but never auto-blocks an OT device.
Showing SIEM export	Syslog export integrates with Splunk, IBM QRadar, or Wazuh without changing existing SOC workflows.

Showing Redis persistence	Even after a restart, SmartShield remembers all evidence and blocked IPs — critical for 24/7 factories.
---------------------------	---

Common Problems and Fast Fixes

Problem	Fix
"Permission denied" on Docker	<code>sudo usermod -aG docker \$USER</code> , then log out and back in.
Web interface shows nothing	Wait 30s. <code>docker ps</code> — is <code>smartshield-web</code> Up? Then <code>docker logs smartshield-web</code> .
Containers keep restarting / "Exiting"	<code>docker logs smartshield</code> — most common cause is a missing <code>docker/.env</code> .
No alerts after attacks	Evidence groups into time windows. Lower <code>evidence_detection_threshold</code> to 0.05 in <code>smartshield.yaml</code> , then restart.
SYN flood doesn't trigger fast enough	For a snappier demo, lower <code>connection_min_flood_packets</code> (default 200) and <code>packet_rate_threshold</code> (default 50) in <code>smartshield.yaml</code> , then restart.
"Port 502 permission denied"	Port 502 needs root — run the fake PLC with <code>sudo python3 ...</code> .
"pymodbus not found"	<code>pip3 install pymodbus --break-system-packages</code>
nmap says "0 open ports"	Start the fake PLC first (Terminal 1) — it must print "Fake PLC running on port 502...".
alerts.json is empty	SmartShield needs a few minutes to start fully. <code>docker logs smartshield grep "Started"</code> .

If all else fails — full reset

```
docker compose -f docker/docker-compose.factory.yml down
sudo rm -rf /var/lib/smartshield/*
sudo mkdir -p /var/lib/smartshield/{redis,output}
sudo chmod 777 /var/lib/smartshield/{redis,output}
docker compose -f docker/docker-compose.factory.yml up -d
```

Cheat Sheet — All Commands in Order

Initial setup (run once)

```
cd ~/SmartShield-Factory
sudo mkdir -p /var/lib/smartshield/{redis,output}
sudo chmod 777 /var/lib/smartshield/{redis,output}
docker build -f docker/Dockerfile -t smartshield_gp:latest .
```

Start SmartShield

```
cd ~/SmartShield-Factory
docker compose -f docker/docker-compose.factory.yml up -d
sleep 10 && docker compose -f docker/docker-compose.factory.yml ps
# Then open: http://localhost:55000
```

Demo attacks (in order)

```
# 1 – Port scan
nmap -sV -p 502,102,20000 127.0.0.1

# 2 – Fake PLC (Terminal 1)
sudo python3 -c "from pymodbus.server import StartTcpServer; from pymodbus.datastore import
ModbusSlaveContext,ModbusServerContext,ModbusSequentialDataBlock;
s=ModbusSlaveContext(co=ModbusSequentialDataBlock(0,[17]*100),hr=ModbusSequentialDataBlock(0,
[17]*100),ir=ModbusSequentialDataBlock(0,[17]*100),di=ModbusSequentialDataBlock(0,[17]*100));
print('Fake PLC on port 502');
StartTcpServer(ModbusServerContext(slaves=s,single=True),address=('0.0.0.0',502))"

# 2 – Flood (Terminal 2)
python3 -c "from pymodbus.client import ModbusTcpClient; import time;
[ (ModbusTcpClient('127.0.0.1',port=502).connect(), time.sleep(0.05)) for _ in range(50) ]"

# 3 – Unauthorized write (Terminal 2)
python3 -c "from pymodbus.client import ModbusTcpClient; c=ModbusTcpClient('127.0.0.1',port=502);
c.connect(); c.write_coil(0,True); c.write_register(1,9999); print('Done')"
```

```
# 4 – SYN flood (new volumetric detector)
sudo hping3 -S -p 502 --flood 127.0.0.1
```

Check results

```
cat /var/lib/smartshield/output/alerts.json
docker logs smartshield --tail 30
sudo iptables -L smartshieldBlocking -n 2>/dev/null
```

Stop everything

```
docker compose -f docker/docker-compose.factory.yml down
# Kill the fake PLC: Ctrl+C in Terminal 1
```